

# PROJECT 6

## Simple Neural Network Forward Propagation

*Using the Iris Classification Dataset*

### 1. Objective

The objective of this project is to understand the basic working of a neural network through forward propagation using the Iris classification dataset. The project demonstrates how input features are passed through the neural network, processed using weights, biases, and activation functions, and finally converted into predicted Iris flower classes.

### 2. Introduction

A Neural Network is a machine learning model inspired by the structure of the human brain. It consists of interconnected layers of neurons that process input data and generate an output.

In this project, the Iris dataset is used for classification. The dataset contains measurements of Iris flowers belonging to three different species: Iris Setosa, Iris Versicolor, and Iris Virginica.

The four input features are:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

### 3. Dataset Description

The Iris dataset is a commonly used dataset for machine learning classification problems.

| Property           | Value |
|--------------------|-------|
| Total Samples      | 150   |
| Number of Features | 4     |
| Number of Classes  | 3     |
| Samples per Class  | 50    |

Target Classes:

| Class | Flower          |
|-------|-----------------|
| 0     | Iris Setosa     |
| 1     | Iris Versicolor |
| 2     | Iris Virginica  |

- Python
- NumPy
- Scikit-learn
- Iris Dataset
- Jupyter Notebook / Google Colab / VS Code

## 5. Task 1 – Load and Prepare the Iris Dataset

The Iris dataset was loaded using the `load_iris()` function from Scikit-learn. The dataset was divided into input features (X) and target labels (y). The data was then divided into training and testing sets using an 80:20 split. The input features were standardized using `StandardScaler` so that the features have a similar scale.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```

# Project 6: Simple Neural Network Forward Propagation

# Import required libraries
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# -----
# Task 1: Load and Prepare the Iris Dataset
# -----

# Load Iris dataset
iris = load_iris()

X = iris.data      # Input features
y = iris.target    # Target classes

print("Dataset Shape:", X.shape)
print("Feature Names:", iris.feature_names)
print("Target Names:", iris.target_names)

# Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Standardize the input features

```

Figure 2: Loading, splitting, and standardizing the Iris dataset.

## 6. Task 2 – Build a Simple Neural Network

A basic neural network was created with an input layer, a hidden layer, and an output layer. The input layer contains 4 neurons corresponding to the four Iris features. The hidden layer contains 5 neurons. The output layer contains 3 neurons corresponding to the three Iris flower classes.

| Layer        | Number of Neurons |
|--------------|-------------------|
| Input Layer  | 4                 |
| Hidden Layer | 5                 |
| Output Layer | 3                 |

```
np.random.seed(42)
```

```
W1 = np.random.randn(4, 5) * 0.1
b1 = np.zeros((1, 5))
```

```
W2 = np.random.randn(5, 3) * 0.1
b2 = np.zeros((1, 3))
```

```

# -----
# Task 2: Build a Simple Neural Network
# -----

# Network structure:
# Input Layer -> Hidden Layer -> Output Layer
# 4 neurons      5 neurons      3 neurons

np.random.seed(42)

# Initialize weights and biases
W1 = np.random.randn(4, 5) * 0.1
b1 = np.zeros((1, 5))

W2 = np.random.randn(5, 3) * 0.1
b2 = np.zeros((1, 3))

# -----
# Task 3: Activation Functions
# -----

# ReLU activation function
def relu(x):
    return np.maximum(0, x)

# Softmax activation function
def softmax(x):
    exp_values = np.exp(x - np.max(x, axis=1, keepdims=True))
    return exp_values / np.sum(exp_values, axis=1, keepdims=True)

```

Figure 3: Initialization of neural network weights and biases.

## 7. Task 3 – Activation Functions

Activation functions are used in neural networks to introduce non-linearity into the model. ReLU was used in the hidden layer and Softmax was used in the output layer.

### 7.1 ReLU Activation Function

The ReLU (Rectified Linear Unit) function is defined as:

$$\text{ReLU}(x) = \max(0, x)$$

### 7.2 Softmax Activation Function

The Softmax function converts the output values into probabilities for the three Iris classes. The class having the highest probability is selected as the predicted class.

## 8. Forward Propagation

Forward propagation is the process of passing input data from the input layer through the hidden layer and finally to the output layer.

$$\mathbf{Z}_1 = \mathbf{X}\mathbf{W}_1 + \mathbf{b}_1$$

$$\mathbf{A}_1 = \text{ReLU}(\mathbf{Z}_1)$$

$$\mathbf{Z}_2 = \mathbf{A}_1\mathbf{W}_2 + \mathbf{b}_2$$

$$\mathbf{A}_2 = \text{Softmax}(\mathbf{Z}_2)$$

The class with the highest probability is selected as the predicted class.

```
def relu(x):  
    return np.maximum(0, x)  
  
def softmax(x):  
    exp_values = np.exp(x - np.max(x, axis=1, keepdims=True))  
    return exp_values / np.sum(exp_values, axis=1, keepdims=True)  
  
Z1 = np.dot(X_test, W1) + b1  
A1 = relu(Z1)  
  
Z2 = np.dot(A1, W2) + b2  
A2 = softmax(Z2)  
  
predictions = np.argmax(A2, axis=1)
```

```

# -----
# Forward Propagation
# -----

# Hidden layer calculation
Z1 = np.dot(X_test, W1) + b1

# Apply ReLU activation
A1 = relu(Z1)

# Output layer calculation
Z2 = np.dot(A1, W2) + b2

# Apply Softmax activation
A2 = softmax(Z2)

# Predicted class
predictions = np.argmax(A2, axis=1)

# -----
# Display Results
# -----

print("\nForward Propagation Output:")
print("-----")

for i in range(10):
    print(
        "Actual:",
        iris.target_names[y_test[i]],
        "| Predicted:",
        iris.target_names[predictions[i]],
        "| Probabilities:",

```

*Figure 4: Forward propagation calculations using ReLU and Softmax.*

## 9. Predicted Output

After forward propagation, the network produces probability values for the three Iris classes. The predicted class is obtained using the `argmax()` function.

```
predictions = np.argmax(A2, axis=1)
```

The output displays the actual class, predicted class, and probability of each class.

Example output:

Forward Propagation Output:

-----  
Actual: versicolor | Predicted: setosa  
Probabilities: [0.34 0.33 0.33]

Actual: setosa | Predicted: setosa  
Probabilities: [0.35 0.33 0.32]

Note: Since the neural network uses randomly initialized weights and is not trained, the predictions are only demonstrations of forward propagation and are not expected to have high classification accuracy.

```
Dataset Shape: (150, 4)
Feature Names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Target Names: ['setosa' 'versicolor' 'virginica']

Forward Propagation Output:
-----
Actual: versicolor | Predicted: setosa | Probabilities: [0.333 0.333 0.333]
Actual: setosa | Predicted: virginica | Probabilities: [0.319 0.312 0.369]
Actual: virginica | Predicted: versicolor | Probabilities: [0.33 0.338 0.333]
Actual: versicolor | Predicted: versicolor | Probabilities: [0.333 0.334 0.333]
Actual: versicolor | Predicted: virginica | Probabilities: [0.332 0.332 0.336]
Actual: setosa | Predicted: virginica | Probabilities: [0.324 0.312 0.364]
Actual: versicolor | Predicted: setosa | Probabilities: [0.333 0.333 0.333]
Actual: virginica | Predicted: versicolor | Probabilities: [0.33 0.338 0.333]
Actual: versicolor | Predicted: virginica | Probabilities: [0.333 0.332 0.335]
Actual: versicolor | Predicted: virginica | Probabilities: [0.333 0.333 0.334]
```

Figure 5: Predicted Iris classes and corresponding probabilities.

## 10. Result

The simple neural network was successfully implemented using Python and the Iris dataset. Forward propagation was performed through the stages of weighted sums, biases, ReLU activation, output calculation, Softmax activation, and class prediction.

## 11. Conclusion

This project demonstrated the basic working of forward propagation in a neural network using the Iris classification dataset. The experiment helped in understanding how input data moves through different neural network layers and how activation functions transform the data. ReLU was used in the hidden layer and Softmax was used in the output layer to produce classification probabilities. Although the network was not trained, the project successfully demonstrated the fundamental concept of how a neural network processes input data and generates predictions.

## 12. Learning Outcomes

- Loading and preparing a machine learning dataset.
- Understanding input features and target classes.

- Understanding the basic structure of a neural network.
- Initializing weights and biases.
- Understanding forward propagation.
- Understanding ReLU activation.
- Understanding Softmax activation.
- Generating classification predictions.
- Understanding how neural networks proc